

Mortgage Engine Code Walkthrough

A maintainer's guide to the mortgage calculation flow, the field-presence solver, APR and APR-crossover comparison, and the underlying financial mathematics.

Scope. The mortgage calculation engine (`Mortgage.pas`), the screen layer and APR/row-generation dialogs that feed it (`MortgageScreenUnit.pas` and the `APR* / SelectAPRDlg / MortgageRowGenerationDlg` units), and the shared interest mathematics (`INTSUTIL.pas`).

Audience. Engineers maintaining or porting the financial logic. Every claim is anchored to a `file:line` reference for verification.

Authority. The DOS source is the authoritative implementation of the financial logic. Line numbers cite the canonical tree `legacy/src/dos_source/`; the annotated mirror `legacy/src_documented/dos_source/` carries inline commentary on the same routines (its line numbers are shifted by the added comments).

Companion. This document follows the same structure as `amortization_dos_walkthrough.pdf`. The amortization, present-value, and actuarial flows share the same field-presence-dispatch pattern documented here.

Contents

0 How to read this document

1 Architecture & data model

1.1 The screen/engine split · 1.2 The mortgageline record · 1.3 The internal rate convention

2 The UI layer: cell → record + status

2.1 Grid & columns · 2.2 The blank-vs-typed mapping · 2.3 Mutual exclusions & validation · 2.4 The calc trigger

3 The core Calc dispatch

3.1 Summation · 3.2 ComputeCashPctAndFinanced · 3.3 Price ↔ Monthly · 3.4 BalloonCalc

4 Balloon & early-termination math

5 APR: the held-to-term yield

6 APR comparison & the crossover solver

7 Row generation (what-if tables)

8 A worked trace

9 Maintainer notes & known quirks

10 Appendix: call graph & file:line index

0 How to read this document

The Per%Sense mortgage screen sizes a loan. Like every Per%Sense screen it uses *field-presence dispatch*: the user fills in what they know, leaves one field blank, and the engine solves for the blank. Leave the monthly payment blank to size a payment; leave the price blank to find the most expensive house a payment can buy; leave a balloon amount blank to size a balloon. On top of the single-loan calc sit two comparison features: a single-loan **APR** (the true yield once points are folded in) and a two-loan **APR crossover** (the holding period at which a low-rate/high-points loan stops beating a high-rate/low-points loan).

Citation convention

Inline references look like `Mortgage.pas:192`. Unless a range is given, the number is the line where the construct begins. **All line numbers refer to the canonical tree** `legacy/src/dos_source/`. The annotated mirror `legacy/src_documented/dos_source/` has the same code with inline comments (shifted line numbers) and is meant for reading alongside this document.

The cast of files

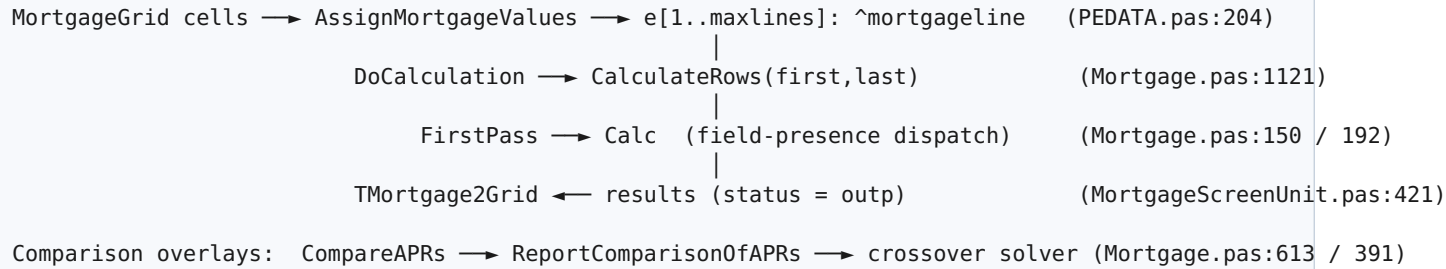
File	Role
<code>Mortgage.pas</code>	Engine. The single-row Calc dispatch, the Summation annuity factor, the APR and crossover solvers, and row generation.
<code>MortgageScreenUnit.pas</code>	UI. The grid, cell↔record marshalling, validation, the calc trigger, and the CompareAPRs / GenerateMtgRows entry points.
<code>SelectAPRDlgUnit</code> / <code>APRComparisonDLGUnit</code>	Pick the second loan to compare; display the comparison result.
<code>APROptionsDlgUnit</code> / <code>APRReportScreenUnit</code>	Collect cost adjustments and render the Reg-Z (Truth-in-Lending) APR report box.
<code>MortgageRowGenerationDlgUnit</code>	Collect the “vary this column” parameters for what-if table generation.
<code>INTSUTIL.pas</code>	Shared math: <code>exxp/lnn</code> , <code>YieldFromRate/RateFromYield</code> , <code>RealPerYr</code> . (Documented in detail in the amortization walkthrough, §5.)

The two things to internalize. (1) Summation (§3.1) is the annuity factor at the heart of every mortgage formula — the present value of n monthly payments of \$1. (2) The stored rate is a *continuously-compounded* rate, never the displayed percentage; the screen converts to/from the 12-period yield at the boundary. Miss either and every number drifts.

1 Architecture & data model

1.1 The screen/engine split

The mortgage feature is two layers over a shared model. The screen (`MortgageScreenUnit.pas`) owns a grid of cells and marshals them into a global array of `mortgageline` records; the engine (`Mortgage.pas`) reads those records, solves the blank field, and writes results back with an `outp` status. The screen never computes; the engine never decides what the user sees.



1.2 The `mortgageline` record

Every loan row is a `mortgageline` (alias `mtgline`) record [PETYPES.PAS:535–564](#). Each numeric field is paired with an `inout` status field recording its provenance (empty=0 blank, `outp`=1 computed, `defp`=2 defaulted, `inp`=3 user-typed — [PETYPES.PAS:137–143](#)):

Field	Screen label	Financial meaning
price	Price	Purchase price of the property.
points	Pts	Origination points as a fraction (0.02 = 2 points), charged on the financed amount.
pct	% Down	Down-payment fraction of price; 1-pct is the loan-to-value.
cash	Cash Required	Cash out of pocket = down payment + points = $\text{price} \cdot (\text{pct} + (1-\text{pct}) \cdot \text{points})$.
financed	Amount Borrowed	Loan principal = $\text{price} \cdot (1-\text{pct})$.
years	Yrs	Term in years (integer).
rate	Rate	The true continuously-compounded rate (see §1.3). Displayed as a 12-period yield.
tax	Monthly Tax+Ins	Flat monthly escrow add-on — <i>not</i> amortized. The amortizing payment is <code>monthly-tax</code> .
monthly	Monthly Total	Total monthly payment = amortizing P&I + tax.
when / howmuch	Balloon Yrs / Amount	A balloon lump of <code>howmuch</code> due when <code>years</code> out.
balloonstatus	—	Derived enum (<code>balloon_known</code> , <code>balloon_unk</code> , <code>balloon_blank</code>) set by <code>FirstPass</code> (§3).

The grid columns are not one-to-one with fields: the three columns % Down / Cash Required / Amount Borrowed map to the *mutually exclusive* pct / cash / financed triad (§2.3). The global array is `e: mtgarray` `PEDATA.pas:204`, allocated per-row by the screen constructor. `ZeroMortgage` `Mortgage.pas:74` clears a row; `MortgageIsEmpty` `Mortgage.pas:101` tests whether all eleven status fields are empty.

1.3 The internal rate convention

rate is a continuous rate, not the displayed percentage. Internally every formula uses $e^{\pm \text{rate} \cdot t}$. The screen converts on the way in — `rate := RateFromYield(typed%/100, 12)` `MortgageScreenUnit.pas:337` — and on the way out — `YieldFromRate(rate, 12) × 100` `MortgageScreenUnit.pas:464`. Monthly compounding ($n=12$) is hard-coded throughout the mortgage unit. A port that stores the nominal yield in `rate` corrupts every formula.

The two conversions are exact inverses `INTSUTIL.pas:1263,1270`: $\text{YieldFromRate}(rr, n) = n \cdot (e^{rr/n} - 1)$ and $\text{RateFromYield}(yy, n) = n \cdot \ln(1 + yy/n)$. See the amortization walkthrough §5.6 for the full treatment.

2 The UI layer: cell → record + status

The form is TMortgageScreen `MortgageScreenUnit.pas:37`, hosting a single MortgageGrid `:38` whose rows are loan records.

2.1 Grid & columns

Column indices are named constants `MortgageScreenUnit.pas:23–33` (column 0 is the row-number gutter):

```
PriceCol=1  PointsCol=2  PercentDownCol=3  CashRequiredCol=4  AmountBorrowedCol=5
YearsCol=6  LoanRateCol=7  MonthlyTaxCol=8  MonthlyTotalCol=9  BalloonYearsCol=10  BalloonAmountCol=11
```

The authoritative store is the global `e: mtgarray` `PEDATA.pas:204`; the grid is a view kept in sync by themarshallers below.

2.2 The blank-vs-typed mapping

AssignMortgageValuesEx `MortgageScreenUnit.pas:258–390` (wrapped by AssignMortgageValues with Hardness=inp) maps a cell to a field + status. Every column uses the same blank-vs-typed shape — here is Price `:265–275`:

```
if( Value = '' ) then begin
  e[ARow+1].Price := 0;
  e[ARow+1].PriceStatus := empty;           { BLANK -> the solve request }
  MortgageGrid.SetCellHardness( ACol, ARow, empty );
end else begin
  e[ARow+1].Price := StringFormat2Double( Value, IsError );
  e[ARow+1].PriceStatus := Hardness;       { typed -> inp }
end;
```

Conversions on the way in: Points and % Down are stored as fractions (/100, `:282,293`); the loan rate becomes a true rate (RateFromYield(%/100, 12), `:337`); Years/When use StringFormat2Int; Tax is stored raw. The inverse TMortgage2Grid `:421–494` formats each field back (Points/Pct × 100, rate via YieldFromRate×100) and colours the cell by status, so a computed (outp) value reads as such. Note the 1-based record vs 0-based grid: `e[ARow+1]` everywhere.

2.3 Mutual exclusions & validation

The down-payment triad is mutually exclusive. MortgageGridCellBeforeEdit

`MortgageScreenUnit.pas:229–249` clears the other two of % Down / Cash / Amount-Borrowed (cells *and* records) when any one is entered, so the engine never sees a contradictory triad. The same exclusion is re-imposed on Harden `:577–591`.

MortgageGridVerifyCellString `:1202–1250` bounds the raw (pre-÷100) inputs: points in [0,10), % down in [–9,100), years in [0,100), rate in (–100,100), balloon years in [0,99). The engine's own FirstPass adds semantic checks: years ≤ 0, a balloon amount given without a date, and financed > price all raise errors

`Mortgage.pas:160–183`.

2.4 The calc trigger

Calculate (or Enter in a cell) flushes the pending edit and calls DoCalculation(Row)

`MortgageScreenUnit.pas:410–418`, which publishes the used-row count and invokes the engine for that one row:

```
nlines[MTGBlock] := MortgageGrid.GetUsedRowCount();  
CalculateRows( Row+1, Row+1 ); { Mortgage.pas:1121 }  
TMortgage2Grid( e[Row+1], MortgageGrid, Row ); { results back, status=outp }
```

Harden promotes a computed cell to `inp` (so the next calc treats it as given); Undo/Redo snapshot the whole record array.

3 The core `calc` dispatch

`FirstPass(row)` `Mortgage.pas:150` validates and derives `balloonstatus` (both `When+HowMuch` typed → `balloon_known`; `When` only → `balloon_unk`, solve it; `HowMuch` without `When` → error). Then `Calc(row)` `Mortgage.pas:192` does the field-presence dispatch.

3.1 Summation — the annuity factor

`Summation(ei, r, t)` `Mortgage.pas:124–143` is the present value, at loan date, of $12 \cdot t$ monthly \$1 payments discounted at continuous rate r :

```
if (abs(r) < teeny) then summation := 12 * t      { zero-rate limit }
else begin
  last := exp(-r * t);          { e(-r*t) }
  f := exp(-r * twelfth);      { e(-r/12) one-month discount }
  summation := f * (1 - last) / (1 - f);
end;
```

$$S(r,t) = f \cdot (1 - e^{-r \cdot t}) / (1 - f) = \sum_{k=1}^{12t} f^k, \quad f = e^{-r/12}$$

A geometric series of $n = 12 \cdot t$ monthly discount factors, degenerating to the payment count $12 \cdot t$ as $r \rightarrow 0$ (the teeny branch). This factor converts a level amortizing payment (monthly-tax) to/from a present value.

3.2 `ComputeCashPctAndFinanced`

Given `Price` known and exactly one of `Pct/Cash/Financed` typed, this nested proc `Mortgage.pas:198–245` derives the other two (status outp):

```
cash = price · (pct + (1-pct) · points) · financed = price · (1-pct)
from cash: pct = (cash/price - points) / (1 - points) · from financed: pct = 1 - financed/price
```

A $\text{pct} \geq 0.995$ result ($\approx 100\%$ down) is nonsensical and poked as the error sentinel `:217,232`.

3.3 Price ↔ Monthly

With the down-payment triad, `Years` and `Rate` all known, the main body `Mortgage.pas:259–310` solves whichever of `Price/Monthly` is blank. First it values any known balloon: `balloonval := howmuch · e-rate · when` (its PV) `:268–270`. Then:

Known	Blank → solved	Formula	Line
Price + Pct/Cash/Financed	Monthly	$\text{monthly} = (\text{price} \cdot (1 - \text{pct}) - \text{balloonval}) / S(\text{rate}, \text{years}) + \text{tax}$	287
Monthly + Pct	Price	$\text{price} = ((\text{monthly} - \text{tax}) \cdot S + \text{balloonval}) / (1 - \text{pct})$	294
Monthly + Cash	Price	$\text{price} = \text{cash} + (1 - \text{points}) \cdot ((\text{monthly} - \text{tax}) \cdot S + \text{balloonval})$	296

Both `Price` and `Monthly` given (with no unknown balloon) is over-determined → the “leave `PRICE` or `MONTHLY PAYMENT` or `ANY BALLOON AMOUNT` blank” error `:280`. The flat tax is added after amortization, never financed.

3.4 BalloonCalc

When the balloon amount is the blank (balloon_unk) and Price+Monthly are both given, BalloonCalc Mortgage.pas:247–257 sizes it: the principal not retired by the regular payments, carried forward to the balloon date.

$$\text{residual}_{PV} = \text{price}(1 - \text{pct}) - (\text{monthly} - \text{tax}) \cdot S(\text{rate}, \text{years}) \rightarrow \text{howmuch} = \text{residual}_{PV} \cdot e^{\text{rate} \cdot \text{when}}$$

Deliberate non-promotion. BalloonCalc leaves balloonstatus := balloon_known commented out :255 so that running CALC twice doesn't reclassify the row as having a *known* balloon (which would change the dispatch on the second pass). A faithful port must not promote the status after solving the balloon.

4 Balloon & early-termination math

Two functions value a loan that is paid off early — the basis of the APR solvers.

TerminalBalloon(ei, t) `Mortgage.pas:312–327` is the remaining balance at time t years (including the payment due that date):

$$TB(t) = [\text{financed} - (\text{monthly-tax}) \cdot S(\text{rate}, t-1/12) - [\text{balloon paid?}] \cdot \text{howmuch} \cdot e^{-\text{rate} \cdot \text{when}}] \cdot e^{\text{rate} \cdot t}$$

The bracket is the present-value residual after subtracting the PV of payments made through t (note $t-1/12$: payments are due at period ends) and any balloon already paid; multiplying by $e^{\text{rate} \cdot t}$ carries it forward to the payoff date.

ValueOfPaymentsForTerminatedLoan(ei, r, t) `Mortgage.pas:329–346` is the value, as of loan date and discounted at an *arbitrary* rate r , of all cash the lender receives if the loan terminates at t :

$$V(r,t) = (\text{monthly-tax}) \cdot S(r, t-1/12) + [\text{balloon before } t] \cdot \text{howmuch} \cdot e^{-r \cdot \text{when}} + [t \leq \text{years}] \cdot TB(t) \cdot e^{-r \cdot t}$$

Discounting the terminal payoff at the *trial* rate r (not the note rate) is exactly what lets the APR solvers search for the yield.

5 APR: the held-to-term yield

The APR is the rate at which the lender's discounted cash inflows equal the net cash advanced after points. `IterateToFindAPRofTerminatedLoan(ei, t, var apr)` `Mortgage.pas:349–384` is a one-variable secant solver:

```
target := financed * (1 - points);           { net cash advanced after points }
apr    := rate + points / years;             { first guess }
repeat
  value := ValueOfPaymentsForTerminatedLoan(ei, apr, t);
  denom := value - oldvalue;
  if (abs(denom) > teeny) then
    newdelta := (target - value) * delta / denom; { secant step }
    apr := apr + newdelta; ...
until (count = 20) or (abs(delta) < teeny);
apr := YieldFromRate(apr, 12);               { report as 12-period yield }
```

- **Target** `:359`: $\text{financed} \cdot (1 - \text{points})$ — the lender's net outlay, since the borrower pays points up front.
- **Convergence** `:380–381`: at most 20 iterations; success requires $|\text{delta}| < \text{tiny}$.
- **Reporting** `:382`: the converged continuous rate is reported as the 12-period yield.

`IterateToFindAPR` `Mortgage.pas:386` calls this with $t = \text{years} + 1/12$ — terminating one month past maturity, i.e. the full-term APR. `OneMonthAPR` `Mortgage.pas:557` gives the worst case (pay off at the first payment):

$\text{apr_rate} := 12 \cdot (1 + \text{yield}/12) / (1 - \text{points})$ `:563` (the “corrected 2/94” points denominator).

`EnoughDataForAPR` `Mortgage.pas:571` gates on `financed+monthly+rate+years` all present; `ReportAPR` `:538` formats the result or reports non-convergence.

6 APR comparison & the crossover solver

A **crossover** is the holding period at which two competing mortgages have equal effective APR. Below it one loan is cheaper; above it the other. Typically a low-rate/high-points loan beats a high-rate/low-points loan only if held long enough to amortize the points — the crossover is that break-even horizon.

6.1 The comparison flow

ReportComparisonOfAPRs(row1, row2, ...) Mortgage.pas:613–711 drives it: solve both rows, compute each full-term APR (IterateToFindAPR) and each one-month APR (OneMonthAPR), then test dominance. If one loan is better at *both* the short and long horizons it is “always better” (apr_crossover := 0, :673,679); otherwise the APR ordering must flip somewhere in between, and the crossover solver finds it :681.

6.2 The two-variable Newton solve

IterateToFindCrossoverAPRAndTime(e1, e2, var apr, var t) Mortgage.pas:391–535 solves simultaneously for the crossover rate r and time t at which both loans reproduce their true value. The objective per loan is normalized to zero:

$$\text{Target}(e) = 1 - V(r, t) / (e.\text{financed} \cdot (1 - e.\text{points}))$$

The nested OneIteration :413–460 builds a 2×2 Jacobian by finite differences (perturbing r by tiny and t by small), then takes a Newton step via Cramer's rule:

$$\begin{aligned} \det &= \partial T_1 / \partial t \cdot \partial T_2 / \partial r - \partial T_1 / \partial r \cdot \partial T_2 / \partial t \\ dr &= (\partial T_2 / \partial t \cdot T_1 - \partial T_1 / \partial t \cdot T_2) / \det, \quad dt = (-\partial T_2 / \partial r \cdot T_1 + \partial T_1 / \partial r \cdot T_2) / \det \end{aligned}$$

First guess: $t := 0.25 \cdot (e1.\text{years} + e2.\text{years})$, $r := \text{RateFromYield}(\frac{1}{2}(\text{apr1} + \text{apr2}), 12)$:513–517. Iterate up to 40 times until both targets are within teeny :522–527; on success report $\text{apr} := \text{YieldFromRate}(r, 12)$ and round t down to the previous whole month :532–533.

Balloon-date discontinuities. ValueOfPaymentsForTerminatedLoan is discontinuous at a balloon date (the when $< t$ test flips). If the Newton iteration stalls, TryBalloonDates Mortgage.pas:462–508 checks each balloon date directly: if the APR ordering flips across that date, the crossover is pinned there. A faithful port needs this special case or it will fail to converge on balloon loans.

6.3 The dialog chain

From the screen, CompareAPRs MortgageScreenUnit.pas:713–794 branches on the valid-row count (NumberOfValidRowsInArray, Mortgage.pas:581): 0 → prompt for data; 1 → single-loan ReportAPR; 2 → compare directly; >2 → pop SelectAPRDlg SelectAPRDlgUnit.pas:32,46 to pick the partner. The result strings feed APRComparisonDLG APRComparisonDLGUnit.pas:28,33,38. A separate Reg-Z disclosure box is produced by APRReportScreen.CreateReport APRReportScreenUnit.pas:177 via APROptionsDlg APROptionsDlgUnit.pas:33,40.

```
CompareAPRs (MortgageScreenUnit.pas:713)
└─ NumberOfValidRowsInArray (Mortgage.pas:581)   FirstPass+Calc+EnoughDataForAPR per row
```

```
| 1 → ReportAPR (Mortgage.pas:538)
└ 2+ → [SelectAPRDlg to pick row2] → ReportComparisonOfAPRs (Mortgage.pas:613)
      | IterateToFindAPR x2 (Mortgage.pas:386)
      | dominance test (OneMonthAPR + term APR)
      └ IterateToFindCrossoverAPRandTime (Mortgage.pas:391) 2-var Newton
        └ Target → ValueOfPaymentsForTerminatedLoan (Mortgage.pas:329)
→ APRComparisonDLG.SetAPR1/2String/SetResultString (APRComparisonDLGUnit.pas:28)
```

7 Row generation (what-if tables)

From a fully-solved seed row the screen can generate a grid of variants. EnoughDataForRowGeneration `Mortgage.pas:839` requires the row to have *computed* a Price, Monthly or HowMuch (status outp). GenerateMtgRows `MortgageScreenUnit.pas:796` then collects up to three “vary this column” specs from MortgageRowGenerationDlg `MortgageRowGenerationDlgUnit.pas:124` (each: column, increment, count), makes room, and nested-loops CopyRowWithIncrement `:937` — cloning the seed, adding the increment to the chosen column (with the same /100 / RateFromYield conversions as input), and re-solving each new row via DoCalculation.

The DOS in-place generator is dead code. The original GenerateWhatIfTable wrote directly to screen memory and is `{ifdef 0}`-disabled `Mortgage.pas:849–1048`. The live row-generation surface is CalculateRows + the dialog above; any port reconstructs the behaviour rather than translating that block.

8 A worked trace

Follow a loan: **\$200,000 price, 20% down, 2 points, 30 years, 6% rate, \$250/mo tax, monthly blank** (solve the payment).

#	What happens	Code
1	Cells typed; % Down entered clears Cash/Financed (mutual exclusion). Rate stored as <code>RateFromYield(0.06,12)</code> . Monthly blank → <code>monthlystatus := empty</code> .	<code>MortgageScreenUnit.pas:229,337</code>
2	Enter → <code>DoCalculation</code> → <code>CalculateRows(i,i)</code> .	<code>MortgageScreenUnit.pas:410</code>
3	FirstPass: no balloon → <code>balloon_blank</code> ; <code>financed ≤ price</code> ok.	<code>Mortgage.pas:150</code>
4	Calc: Price typed → <code>ComputeCashPctAndFinanced</code> sets <code>financed = 200000·0.80 = \$160,000</code> , <code>cash = 200000·(0.20+0.80·0.02) = \$43,200</code> .	<code>Mortgage.pas:198</code>
5	Monthly blank, Price known → <code>monthly = 160000 / S(rate,30) + 250</code> . <code>S</code> is the 360-payment annuity factor.	<code>Mortgage.pas:287</code>
6	<code>monthlystatus := outp</code> ; <code>TMortgage2Grid</code> writes the payment back, coloured as computed.	<code>MortgageScreenUnit.pas:421</code>

Change it to leave **Price** blank with a payment given: step 5 instead solves $\text{price} = ((\text{monthly} - \text{tax}) \cdot S + \text{balloonval}) / (1 - \text{pct})$:294. Ask for the loan's APR instead: `CompareAPRs` with one valid row calls `ReportAPR` → `IterateToFindAPR`, which secant-solves the rate that makes the discounted payments equal $\text{financed} \cdot (1 - \text{points})$ and reports it as a 12-period yield.

9 Maintainer notes & known quirks

- **Continuous rate, monthly reporting.** rate is continuous; the screen converts via `RateFromYield/YieldFromRate` with `n=12` hard-coded `MortgageScreenUnit.pas:337,464`. Monthly compounding is assumed throughout — not user-configurable on this screen.
- **The $t-1/12$ offset.** The termination functions use `Summation(..., t-1/12)` `Mortgage.pas:321,337` but the payment solve uses the full term `:287`. Payments are due at period ends; the terminal payoff is valued separately. Easy to drop in a port.
- **Balloon status not promoted.** `BalloonCalc` deliberately leaves the row `balloon_unk` after solving `:255` so a second `CALC` behaves identically.
- **The $pct \geq 0.995$ sentinel.** Out-of-range down payment is poked as `error = -8888` `:217,232` rather than raised immediately; downstream `ok()` checks catch it.
- **OneMonthAPR “corrected 2/94”.** The points-amortization denominator (1-points) `:564` is a preserved bug fix.
- **Crossover needs the balloon-date special case.** Without `TryBalloonDates` `:462` the 2-variable Newton iteration fails to converge when the crossover lands on a balloon date.

10 Appendix: call graph & file:line index

```

CalculateRows (public)                                Mortgage.pas:1121
├─ FirstPass → balloonstatus, validation             Mortgage.pas:150
├─ Calc                                              Mortgage.pas:192
│   ├─ ComputeCashPctAndFinanced Pct←Cash←Financed :198
│   ├─ BalloonCalc → Summation                      :247 / 124
│   └─ payment/price solve → Summation              :287 / 124

ReportAPR → IterateToFindAPR → IterateToFindAPRofTerminatedLoan (secant) :538 / 386 / 349
├─ ValueOfPaymentsForTerminatedLoan :329
│   └─ TerminalBalloon → Summation :312 / 124
└─ YieldFromRate [INTSUTIL:1263]

ReportComparisonOfAPRs                                Mortgage.pas:613
├─ IterateToFindAPR ×2 ; OneMonthAPR (557)
├─ IterateToFindCrossoverAPRAndTime 2-var Newton :391
│   ├─ Target → ValueOfPaymentsForTerminatedLoan :400 / 329
│   ├─ OneIteration Jacobian + Cramer :413
│   └─ TryBalloonDates :462

UI: AssignMortgageValuesEx (258) · TMortgage2Grid (421) · DoCalculation (410) ·
    CompareAPRs (713) · GenerateMtgRows (796) [MortgageScreenUnit.pas]

```

Symbol	Reference
mortgageline record / status codes	PETYPES.PAS:535 / 137–143
Global model e: mtgarray	PEDATA.pas:204
Summation (annuity factor)	Mortgage.pas:124
FirstPass / Calc	Mortgage.pas:150 / 192
ComputeCashPctAndFinanced / BalloonCalc	Mortgage.pas:198 / 247
TerminalBalloon / ValueOfPaymentsForTerminatedLoan	Mortgage.pas:312 / 329
APR solvers / OneMonthAPR	Mortgage.pas:349, 386 / 557
ReportComparisonOfAPRs / crossover	Mortgage.pas:613 / 391
Row generation	Mortgage.pas:839, 1121 · MortgageScreenUnit.pas:796
UI marshalling / calc trigger	MortgageScreenUnit.pas:258, 421 / 410
APR dialogs	SelectAPRDLgUnit:32 · APRComparisonDLGUnit:28 · APRReportScreenUnit:177

Per%Sense DOS→Go port. Line numbers cite legacy/src/dos_source/; the annotated mirror legacy/src_documented/dos_source/ carries inline commentary. Companion documents: amortization_dos_walkthrough.pdf, present_value_dos_walkthrough.pdf, actuarial_dos_walkthrough.pdf.